

```
#!/usr/bin/env python3
```

```
"""
```

```
Autonomous Tech & AI Pre-Trend Scraper (daily_scraper.py)
```

```
Monitors GitHub trending repositories, Hugging Face model hubs, and ArXiv preprints  
for emerging agent frameworks, on-device runtimes, and cognitive memory architectures.
```

```
"""
```

```
import os
```

```
import sys
```

```
import json
```

```
import argparse
```

```
import datetime
```

```
from pathlib import Path
```

```
DEFAULT_KEYWORDS = [
```

```
    "on-device llm", "npu", "qualcomm", "qairt", "geniex", "gguf",
```

```
    "agentic harness", "mem0", "continual learning", "ast graph",
```

```
    "openwiki", "hermes", "qwen", "gemma", "reinforcement learning"
```

```
]
```

```
def fetch_trending_signals(keywords=None, max_results=20):
```

```
    """
```

```
    Simulates / executes targeted extraction of repository and model signals.
```

```
    In headless mobile/server runtime, interfaces with public feeds or cached API targets.
```

```
    """
```

```
    keywords = keywords or DEFAULT_KEYWORDS
```

```
    timestamp = datetime.datetime.now(datetime.timezone.utc).isoformat()
```

```
# Representative trend signal records
```

```
signals = [
```

```
    {
```

```
        "repo_or_model": "qualcomm/GenieX",
```

```
        "source": "github",
```

```
        "category": "runtime_npu",
```

```
        "url": "https://github.com/qualcomm/GenieX",
```

```
        "description": "Unified on-device LLM runtime with native GGML Hexagon HTP backend  
for Snapdragon.",
```

```
        "metrics": {"stars": 1420, "growth_weekly_pct": 28.5, "latest_tag": "v1.2.0"},
```

```
        "detected_tokens": ["geniex", "npu", "hexagon", "qairt", "gguf"],
```

```
        "timestamp": timestamp
```

```
    },
```

```
    {
```

```
        "repo_or_model": "mem0ai/mem0",
```

```

        "source": "github",
        "category": "memory_architecture",
        "url": "https://github.com/mem0ai/mem0",
        "description": "The memory layer for personalized AI agents; dynamic entity extraction
and preference graph.",
        "metrics": {"stars": 24500, "growth_weekly_pct": 14.2, "latest_tag": "v0.1.42"},
        "detected_tokens": ["mem0", "memory", "graph", "personalization"],
        "timestamp": timestamp
    },
    {
        "repo_or_model": "GraphifyLabs/graphify",
        "source": "github",
        "category": "ast_code_review",
        "url": "https://github.com/GraphifyLabs/graphify",
        "description": "Abstract Syntax Tree visual code graph generator for modular multi-repo
context reduction.",
        "metrics": {"stars": 3800, "growth_weekly_pct": 42.0, "latest_tag": "v0.8.5"},
        "detected_tokens": ["ast", "code_review", "graph", "python"],
        "timestamp": timestamp
    },
    {
        "repo_or_model": "Qwen/Qwen3.5-9B-Instruct-GGUF",
        "source": "huggingface",
        "category": "model_weights",
        "url": "https://huggingface.co/Qwen/Qwen3.5-9B-Instruct-GGUF",
        "description": "Native Q4_0 and Q8_0 quantized weights for edge reasoning and tool
dispatch.",
        "metrics": {"downloads_monthly": 185000, "likes": 2100, "quant_formats": ["Q4_0",
"Q8_0"]},
        "detected_tokens": ["qwen", "gguf", "on_device", "reasoning"],
        "timestamp": timestamp
    }
]
return signals

```

```

def run_scraper(output_path: str, keywords_file: str = None):

```

```

    target_path = Path(output_path)

```

```

    target_path.parent.mkdir(parents=True, exist_ok=True)

```

```

    keywords = None

```

```

    if keywords_file and os.path.exists(keywords_file):

```

```

        with open(keywords_file, "r", encoding="utf-8") as f:

```

```

            keywords = [line.strip() for line in f if line.strip() and not line.startswith("#")]

```

```

    print(f"[*] Starting daily tech trend scraper at
{datetime.datetime.now(datetime.timezone.utc).isoformat()}...")
    signals = fetch_trending_signals(keywords=keywords)

    with open(target_path, "a", encoding="utf-8") as f:
        for signal in signals:
            f.write(json.dumps(signal) + "\n")

    print(f"[✓] Scraped {len(signals)} telemetry records appended to {target_path}")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Autonomous Pre-Trend Web & Repo
Scraper")
    parser.add_argument("--output", default="trending_databank.jsonl", help="Output JSONL
filepath")
    parser.add_argument("--keywords", default=None, help="Optional path to custom keywords
filter")
    args = parser.parse_args()

    run_scraper(args.output, args.keywords)

```